

CS-1204

Object-Oriented Programming

(3+1)

Week 09(2)

By

Engr. Muhammad Awais

Contents

- Operator Overloading

Learning Objectives

- Understand what operator overloading is.
- Learn how to overload various operators in C++.
- Identify the link between operator overloading and compile-time polymorphism.
- Explore real-world examples and lab tasks

What is Operator Overloading?

- Operator Overloading is **redefining the behavior of operators for user-defined types**.
- Improves readability and intuitive interaction with objects
- Syntax:

```
returnType operator symbol (arguments) {  
    ... ..  
}
```

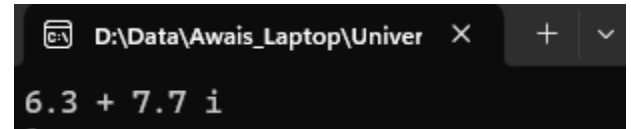
Example of Operator Overloading in C++?

```
class Complex{
public:
    double real,imag;
    Complex(){
    }
    Complex(double real, double imag){
        this->real = real;
        this->imag = imag;
    }
    Complex operator + (Complex c){
        Complex ans;
        ans.real = c.real + this->real;
        ans.imag = c.imag + this->imag;
        return ans;
    }
    void printComplexNumber(){
        cout << this->real << " + " << this->imag << " i" << endl;
    }
};

int main(){

    Complex c1(2.3,3.2), c2(4.0,4.5);
    Complex c3 = c1 + c2;
    c3.printComplexNumber();

    return 0;
}
```



D:\Data\Awais_Laptop\Univer X + v
6.3 + 7.7 i

Compile-Time Polymorphism

- Operator overloading is a type of **compile-time (static) polymorphism**.
- It allows same operator to work differently for user-defined types (e.g., adding two Complex numbers).
- It is resolved during compilation, not at runtime.

Practice Example

Overload == Operator for Distance

- Create a class Distance with two private data members: feet and inches.
- Provide a constructor to initialize the values.
- Overload the == operator using a member function.
- Display whether two distances are equal or not.

Quiz

You are developing a salary calculation system for a company that has two types of employees:

- **Permanent employees** who have a fixed basic salary and a bonus.
- **Contract employees** who are paid based on hourly rate and number of hours worked

What You Have to Do:

- Create a base class **Employee**:
 - It should have a virtual function **calculateSalary()** that prints "Base Employee salary calculation called." and returns 0.
- Create a derived class **PermanentEmployee**:
 - Add members for basicSalary and bonus.
 - **Override calculateSalary()** to return basicSalary + bonus
- Create a derived class **ContractEmployee**:
 - Add members for hourlyRate and hoursWorked.
 - **Override calculateSalary()** to return hourlyRate * hoursWorked
- In the main() function:
 - Use a base class pointer to call calculateSalary() for both derived class objects.
 - Simulate the scenario with these values:
 - PermanentEmployee: basicSalary = 40000, bonus = 5000
 - ContractEmployee: hourlyRate = 800, hoursWorked = 30

References

<https://www.geeksforgeeks.org/cpp/operator-overloading-cpp/>